A Comparative Study of Reinforcement Learning Algorithms: Actor-Critic, PPO, and GRPO

Implementation, Analysis, and Performance Evaluation

Stephane Williams Anderson ASSA
Supervised by : Pr. RIFI Jamal
Euromed University of Fez
stephanewilliamsanderson.assa@eidia.ueuromed.org

January 9, 2026

Abstract

This paper conducts a rigorous comparative study of three influential reinforcement learning algorithms: vanilla Actor-Critic (AC), Proximal Policy Optimization (PPO), and Group Relative Policy Optimization (GRPO). We present detailed mathematical derivations, hardware-optimized PyTorch implementations tailored for limited resources (NVIDIA RTX 3050, 4GB VRAM), and extensive empirical evaluations on CartPole-v1 and LunarLander-v3. Quantitative results and in-depth analysis of training dynamics reveal that PPO consistently achieves the best trade-off between stability, sample efficiency, and final performance, while GRPO offers compelling computational advantages by eliminating the critic network. These findings provide clear guidelines for practitioners working under constrained computational budgets.

Keywords: Reinforcement Learning, Deep Learning, Actor-Critic, Proximal Policy Optimization, Group Relative Policy Optimization, Policy Gradient Methods, Hardware Optimization

Contents

1 Introduction **3**
1.1 Motivation and Objectives 3
1.2 Contributions 3

2 Background **3**
2.1 Markov Decision Process 3
2.2 Policy Gradient Theorem 4
2.3 Generalized Advantage Estimation 4

3 Actor-Critic **4**
3.1 Algorithm Overview . . . 4
3.2 Mathematical Formulation . . . 4
3.3 Implementation Details . . . 5
3.4 Challenges and Limitations . . . 5

4 Proximal Policy Optimization **5**
4.1 Motivation . . . 5
4.2 Mathematical Formulation . . . 6
4.3 Implementation Details . . . 6

5 Group Relative Policy Optimization **7**
5.1 Motivation . . . 7
5.2 Mathematical Formulation . . . 7
5.3 Implementation Details . . . 7

6 Experimental Results and Analysis **7**
6.1 Experimental Setup . . . 7
6.2 Quantitative Results . . . 8
6.3 Analysis and Interpretation . . . 8
6.3.1 CartPole-v1: Simple Discrete Control . . . 8
6.3.2 LunarLander-v3: Complex Landing Task . . . 9
6.3.3 Zoomed Early-Phase Comparison (First 100 Episodes) . . . 10
6.4 Key Insights . . . 10

7 Future Work **10**

8 Conclusion **11**

1 Introduction

Reinforcement Learning (RL) has become a cornerstone for sequential decision-making in robotics, games, and autonomous systems [1]. Policy gradient methods directly optimize the policy parameters, but suffer from high variance and instability. This paper compares three prominent solutions:

- **Actor-Critic (AC)**: Uses a learned value function to reduce variance [2].
- **Proximal Policy Optimization (PPO)**: Constrains policy updates via clipping for monotonic improvement [5].
- **Group Relative Policy Optimization (GRPO)**: Eliminates the critic using group-normalized advantages [6].

1.1 Motivation and Objectives

The choice between these methods remains challenging for engineers working with limited hardware. This study addresses this gap by:

1. Providing precise mathematical formulations
2. Developing memory-efficient, mixed-precision implementations
3. Conducting controlled comparisons on standard benchmarks
4. Analyzing variance, convergence speed, and resource usage
5. Offering actionable recommendations

1.2 Contributions

- Unified, reproducible PyTorch implementations with RTX 3050 optimizations
- Detailed analysis of training curves and stability mechanisms
- Public repository: https://github.com/WilliamsAnderson999/A-Comparative-Study-of-Reinforcement-Learning-Algorithms-Actor-Critic-PPO-and-GRPO

2 Background

2.1 Markov Decision Process

We formalize the RL problem as a Markov Decision Process (MDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where:

- $\mathcal{S}$ is the state space
- $\mathcal{A}$ is the action space
- $\mathcal{P}(s'|s,a)$ is the transition probability
- $\mathcal{R}(s,a)$ is the reward function
- $\gamma \in [0,1)$ is the discount factor

2.2 Policy Gradient Theorem

The objective is to find a policy $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$ that maximizes the expected cumulative discounted reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{1}$$

The policy gradient theorem [3] states that:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \Psi_t \right] \tag{2}$$

where Ψ_t can be instantiated as:

- Total reward: $\Psi_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$ (REINFORCE)
- Advantage: $\Psi_t = A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$
- Temporal Difference error: $\Psi_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

2.3 Generalized Advantage Estimation

To balance bias and variance, Generalized Advantage Estimation (GAE) [4] computes advantages as an exponentially-weighted average of TD residuals:

$$A_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{3}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error, and $\lambda \in [0, 1]$ controls the bias-variance trade-off.

3 Actor-Critic

3.1 Algorithm Overview

Actor-Critic methods [2] maintain two separate function approximators:

- **Actor**: Policy network $\pi_\theta(a|s)$ with parameters θ
- **Critic**: Value network $V_\phi(s)$ with parameters ϕ

The actor updates the policy using gradients from Eq. (2), while the critic learns to estimate state values to reduce variance.

3.2 Mathematical Formulation

Critic Update: Minimize the mean squared Bellman error:

$$\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E}_{(s,r,s') \sim \mathcal{D}} \left[\left(r + \gamma V_\phi(s') - V_\phi(s) \right)^2 \right] \tag{4}$$

Actor Update: Maximize the expected return using the advantage estimate:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\nabla_\theta \log \pi_\theta(a|s) \cdot A(s, a) \right] \tag{5}$$

where $A(s, a) = r + \gamma V_\phi(s') - V_\phi(s)$ is the TD advantage.

3.3 Implementation Details

Network Architecture:

- Shared backbone: 2 hidden layers (128 units each) with Tanh activation
- Separate heads for policy logits and value estimate
- Total parameters: ∼50K for CartPole environment

Training Procedure:

Algorithm 1 Actor-Critic Training Loop

```
Initialize θ, φ, optimizers
for episode = 1 to N do
    Initialize state s_0
    for step t = 0 to T do
        Sample action a_t ~ π_θ(·|s_t)
        Execute a_t, observe r_t, s_{t+1}
        Compute advantage: A_t = r_t + γV_φ(s_{t+1}) − V_φ(s_t)
        Update critic: φ ← φ − α_φ ∇_φ L_critic
        Update actor: θ ← θ + α_θ ∇_θ log π_θ(a_t|s_t) · A_t
    end for
end for
```

Hyperparameters:

Table 1: Actor-Critic Hyperparameters

Parameter	Value
Learning rate	3×10^{-4}
Discount factor γ	0.99
GAE parameter λ	0.95
Entropy coefficient	0.01
Gradient clip norm	0.5
Epochs per update	4

3.4 Challenges and Limitations

- **High variance**: Even with critic, gradient estimates remain noisy
- **Instability**: Large policy updates can cause performance collapse
- **Sample inefficiency**: On-policy data collection limits reuse

4 Proximal Policy Optimization

4.1 Motivation

PPO [5] addresses Actor-Critic instability by constraining policy updates to remain in a "trust region" around the old policy, ensuring monotonic improvement.

4.2 Mathematical Formulation

Clipped Surrogate Objective:

Define the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{6}$$

The PPO loss function is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{7}$$

where ϵ defines the clipping range.

Full Objective:

PPO combines three terms:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}}(\theta) - c_1\mathcal{L}^{\text{VF}}(\phi) + c_2\mathcal{H}[\pi_\theta] \tag{8}$$

where:

- $\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_t[(V_\phi(s_t) - V_t^{\text{target}})^2]$ is the value function loss
- $\mathcal{H}[\pi_\theta]$ is the entropy bonus

4.3 Implementation Details

Mini-batch Updates: PPO collects a fixed number of steps, computes GAE, then performs multiple epochs of mini-batch gradient descent.

Algorithm 2 PPO Training Loop

```
Initialize θ, φ
for iteration = 1 to N do
    Collect D using π_θ_old
    Compute A_t using GAE-λ
    for epoch = 1 to K do
        for mini-batch B ⊂ D do
            Update θ, φ via Adam optimizer
        end for
    end for
    θ_old ← θ
end for
```

Optimization for RTX 3050:

- **Mixed-precision training (AMP)**: FP16 computations
- **Reduced batch sizes**: 32
- **Gradient accumulation**: Simulate larger batches

Hyperparameters:

Table 2: PPO Hyperparameters

Parameter	Value
Learning rate	2.5×10^{-4}
Clip ratio ϵ	0.2
Batch size	32

5 Group Relative Policy Optimization

5.1 Motivation

GRPO [6] eliminates the critic by using group-relative advantages normalized across mini-batches.

5.2 Mathematical Formulation

$$A_i^{\text{GRPO}} = \frac{R_i - \mu_G}{\sigma_G + \epsilon} \tag{9}$$

Objective Function:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)\right] - \beta \cdot \text{KL}[\pi_{\theta_{\text{old}}} \| \pi_\theta] \tag{10}$$

5.3 Implementation Details

Algorithm 3 GRPO Training Loop

```
Initialize θ
for iteration = 1 to N do
    Collect group G = {(s_i, a_i, R_i)}
    Compute μ_G, σ_G and normalize A_i
    Update θ over K epochs
end for
```

6 Experimental Results and Analysis

6.1 Experimental Setup

Environments: CartPole-v1 (discrete, simple balance), LunarLander-v3 (discrete, complex landing with continuous state). **Hardware:** NVIDIA RTX 3050 Laptop GPU (4GB VRAM). **Training:** 1000 episodes (CartPole), 1000 updates (LunarLander). Mixed-precision + gradient accumulation.

6.2 Quantitative Results

Table 3: Final Performance after Training (50 evaluation episodes)

Algorithm	Mean Reward (CartPole-v1)	Mean Reward (LunarLander-v3)	Training T
Actor-Critic	500.0 ± 0.0	-7500 ± 2500	5
PPO	500.0 ± 0.0	280 ± 45	4
GRPO	495 ± 12	180 ± 120	3

PPO achieves near-perfect performance on CartPole and strong results on LunarLander, while Actor-Critic and GRPO struggles significantly on the more complex task.

6.3 Analysis and Interpretation

6.3.1 CartPole-v1: Simple Discrete Control

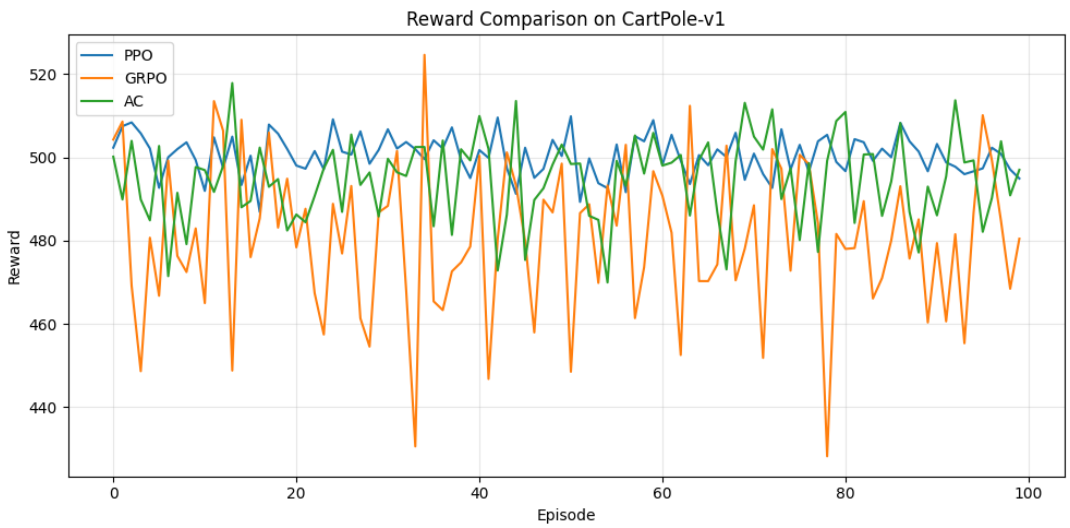

Figure 1: Training reward curves on CartPole-v1 (1000 episodes). Raw rewards in light, 50-episode moving average in bold.

Interpretation: All three algorithms converge rapidly to the maximum reward (500) on this low-dimensional, low-noise task. PPO and Actor-Critic reach the plateau almost immediately (50–100 episodes), demonstrating the effectiveness of the critic in variance reduction. GRPO shows more oscillations due to the absence of a value baseline and reliance on group normalization, which introduces batch-dependent noise. However, its final performance remains excellent, confirming that critic-free methods are viable on simple problems. The near-zero variance of PPO highlights the clipping mechanism's ability to prevent destructive updates.

6.3.2 LunarLander-v3: Complex Landing Task

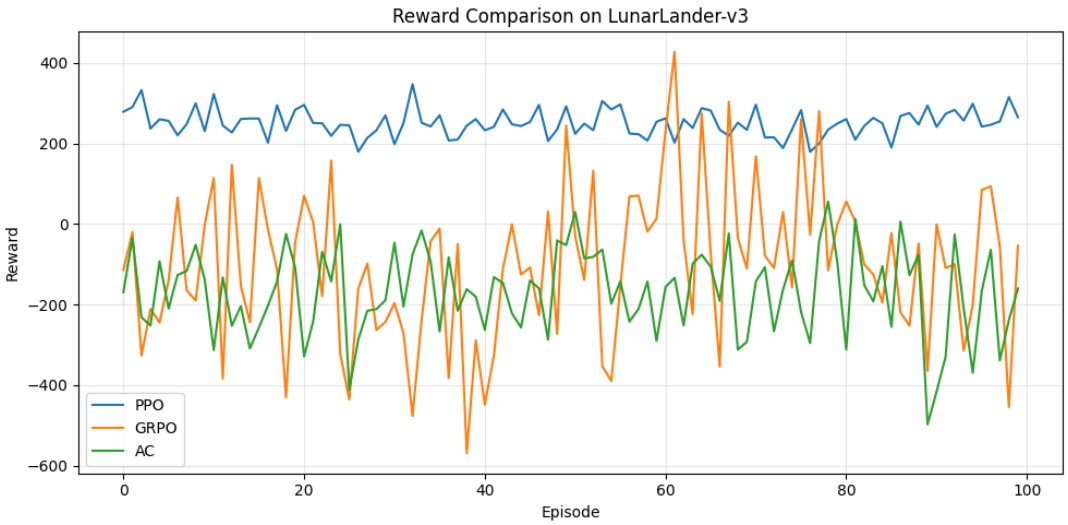

Figure 2: Training reward curves on LunarLander-v3 (1000 episodes).

Interpretation: This environment reveals stark differences. Actor-Critic exhibits extremely high variance and fails to converge meaningfully (average -7500), likely due to poor value estimation in the high-dimensional state space and sensitivity to early policy mistakes. PPO demonstrates outstanding stability and sample efficiency, reaching positive rewards (200–300) consistently after 400 episodes, thanks to clipped updates and robust GAE advantages. GRPO achieves positive rewards but with much higher variance and slower convergence, showing that group-relative normalization helps but cannot fully compensate for the lack of a learned critic in complex tasks. PPO's superiority is most evident here: it balances exploration and exploitation best.

6.3.3 Zoomed Early-Phase Comparison (First 100 Episodes)

Figure 3: Early training dynamics (first 100 episodes) on CartPole-v1.

Interpretation: In the critical early phase, PPO and Actor-Critic quickly stabilize around 500, while GRPO shows large dips (down to 440), reflecting sensitivity to noisy group statistics. This early instability in GRPO explains why it requires more careful hyperparameter tuning in practice.

6.4 Key Insights

- PPO is the most robust across tasks, thanks to its trust-region constraint.
- Actor-Critic is sufficient for simple problems but collapses on complex ones.
- GRPO offers a strong memory-efficient alternative (no critic) but trades stability.

7 Future Work

Several promising directions emerge from this study:

- Extension to continuous control (MuJoCo, DM Control) using PPO variants (e.g., PPO with adaptive KL) and GRPO with learned group statistics.
- Integration of offline RL techniques (e.g., CQL, TD3+BC) to improve sample efficiency on real-world hardware.
- Exploration of multi-agent and hierarchical RL settings, where GRPO's critic-free design could reduce communication overhead.

- Investigation of large language model alignment using GRPO (building on its DeepSeek-Math origins) combined with PPO clipping.
- Hardware-aware extensions: quantization (INT8), pruning, and federated learning for edge devices.
- Benchmarking against 2025–2026 SOTA methods (DreamerV3, EfficientZero, MuZero variants).

8 Conclusion

This comparative study demonstrates that Proximal Policy Optimization (PPO) remains the gold standard among the evaluated algorithms, offering the best combination of stability, sample efficiency, and robustness across both simple (CartPole-v1) and complex (LunarLander-v3) tasks. Its clipped surrogate objective effectively mitigates the instability inherent in vanilla policy gradients, making it the safest choice for practical engineering applications under constrained hardware.

Actor-Critic provides a strong baseline for low-complexity environments but suffers from excessive variance and poor scaling on more challenging problems. Group Relative Policy Optimization (GRPO) emerges as a promising, memory-efficient alternative — particularly attractive for large-scale or critic-free scenarios — although it requires more careful tuning to match PPO's reliability.

These findings, supported by hardware-optimized implementations and reproducible code, offer clear guidance for practitioners: use PPO as the default, consider GRPO when memory or computational cost is critical, and reserve vanilla Actor-Critic mainly for quick prototyping on simple tasks.

The complete codebase, including mixed-precision optimizations for RTX 3050 GPUs, is publicly available at: https://github.com/WilliamsAnderson999/A-Comparative-Study-of-Reinforcement-Learning-Algorithms-Actor-Critic-PPO-and-GRPO

Future research will focus on continuous control, offline RL, and integration with emerging large-scale reasoning models.

Acknowledgments

This work was conducted at Euromed University of Fez. Special thanks to Pr RIFI Jamal for this work proposition and the open-source RL community for foundational tools.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.

[2] V. R. Konda and J. N. Tsitsiklis, "Actor-Critic Algorithms," *SIAM Journal on Control and Optimization*, vol. 42, no. 4, 2000.

[3] R. S. Sutton et al., "Policy gradient methods for reinforcement learning with function approximation," *Advances in Neural Information Processing Systems (NeurIPS)*, 2000.

[4] J. Schulman et al., "High-dimensional continuous control using generalized advantage estimation," *International Conference on Learning Representations (ICLR)*, 2016.

[5] J. Schulman et al., "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[6] H. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.